Log levels

When working with Camunda 8, you may see various messages in your logs. Not all messages require action.

Understanding log levels

Camunda 8 uses the following log levels:

- TRACE: Information which is helpful only if you want to trace the execution of a particular component.
- DEBUG: Information which can provide helpful context when debugging. You may see a DEBUG message right after an INFO message to provide more context.
- INFO: Information about the system which is useful for the user (in the case of the broker, the user here is the user deploying it). For example, leader changes, a new node added to or removed from the membership, etc.
- WARN: Expected errors (e.g. connection timeouts, the remote node is unavailable, etc.) which may indicate that parts of the system are not working, and would require attention if they persist, but may resolve by themselves. These should be monitored, but may not require a support ticket.
- ERROR: Errors which require a person to look into them, e.g. log corruption, inconsistent log, anything which could shut down a partition, etc.

Enable logging

Zeebe

Logging

Zeebe uses Log4j2 framework for logging. In the distribution and the Docker image, find the default log configuration file in `config/log4j2.xml`.

Google Stackdriver (JSON) logging

To enable Google Stackdriver compatible JSON logging, set the environment variable `ZEEBE_LOG_APPENDER=Stackdriver` before starting Zeebe.

Default logging configuration

- `config/log4j2.xml` (applied by default)

```
<?xml version="1.0" encoding="UTF-8"?>
<Configuration status="WARN" shutdownHook="disable">

  <Properties>
    <Property name="log.path">${sys:app.home}/logs</Property>
    <Property name="log.pattern">%d{yyyy-MM-dd HH:mm:ss.SSS} [%X{actor-
name}] [%t] %-5level %logger{36} - %msg%n</Property>
    <Property
name="log.stackdriver.serviceName">${env:ZEEBE_LOG_STACKDRIVER_SERVICENAME:
-}</Property>
    <Property
name="log.stackdriver.serviceVersion">${env:ZEEBE_LOG_STACKDRIVER_SERVICEVE
RSION:-}</Property>
  </Properties>

  <Appenders>
    <Console name="Console" target="SYSTEM_OUT">
      <PatternLayout
        pattern="${log.pattern}"/>
    </Console>

    <Console name="Stackdriver" target="SYSTEM_OUT">
      <StackdriverLayout serviceName="${log.stackdriver.serviceName}"
        serviceVersion="${log.stackdriver.serviceVersion}" />
    </Console>

    <RollingFile name="RollingFile" fileName="${log.path}/zeebe.log"
      filePattern="${log.path}/zeebe-%d{yyyy-MM-dd}-%i.log.gz">
      <PatternLayout>
        <Pattern>${log.pattern}</Pattern>
      </PatternLayout>
      <Policies>
        <TimeBasedTriggeringPolicy/>
        <SizeBasedTriggeringPolicy size="250 MB"/>
      </Policies>
    </RollingFile>
  </Appenders>

  <Loggers>
    <Logger name="io.camunda.zeebe" level="${env:ZEEBE_LOG_LEVEL:-info}"/>

    <Logger name="io.atomix" level="${env:ATOMIX_LOG_LEVEL:-warn}"/>

    <Root level="info">
      <AppenderRef ref="RollingFile"/>

      <!-- remove to disable console logging -->
      <AppenderRef ref="${env:ZEEBE_LOG_APPENDER:-Console}"/>
    </Root>
  </Loggers>

</Configuration>
```

Change log level dynamically

Zeebe brokers expose a [Spring Boot Actuators web endpoint]() for configuring loggers dynamically. To change the log level of a logger, make a `POST` request to

the `/actuator/loggers/{logger.name}` endpoint as shown in the example below. Change `io.camunda.zeebe` to the required logger name and `debug` to required log level.

```
curl 'http://localhost:9600/actuator/loggers/io.camunda.zeebe' -i -X POST -
H 'Content-Type: application/json'
```

Operate

Logging

Operate uses the Log4j2 framework for logging. In the distribution archive, as well as inside a Docker image, `config/log4j2.xml` logging configuration files are included and can be further adjusted to your needs:

```
<?xml version="1.0" encoding="UTF-8"?>
<Configuration status="WARN" monitorInterval="30">
  <Properties>
    <Property name="LOG_PATTERN">%clr{%d{yyyy-MM-dd HH:mm:ss.SSS}}{faint}
%clr{%5p} %clr{${sys:PID}}{magenta} %clr{---}{faint} %clr{[%15.15t]}{faint}
%clr{%-40.40c{1.}}{cyan} %clr{:}{faint} %m%n%xwEx</Property>
  </Properties>
  <Appenders>
    <Console name="Console" target="SYSTEM_OUT" follow="true">
      <PatternLayout pattern="${LOG_PATTERN}"/>
    </Console>
    <Console name="Stackdriver" target="SYSTEM_OUT" follow="true">
      <StackdriverJSONLayout/>
    </Console>
  </Appenders>
  <Loggers>
    <Logger name="io.camunda.operate" level="info" />
    <Root level="info">
      <AppenderRef ref="${env:OPERATE_LOG_APPENDER:-Console}"/>
    </Root>
  </Loggers>
</Configuration>
```

By default, `ConsoleAppender` is used.

JSON logging configuration

You can choose to output logs in JSON format (Stackdriver compatible). To enable it, define the environment variable `OPERATE_LOG_APPENDER` like this:

```
OPERATE_LOG_APPENDER=Stackdriver
```

Change logging level at runtime

Operate supports the default scheme for changing logging levels as provided by Spring Boot.

The log level for Operate can be changed by following the Setting a Log Level section.

Set all Operate loggers to DEBUG

```
curl 'http://localhost:8080/actuator/loggers/io.camunda.operate' -i -X POST
\
-H 'Content-Type: application/json' \
-d '{"configuredLevel":"debug"}'
```

Example of application.yml file

The following snippet represents the default Operate configuration, which is shipped with the distribution. This can be found inside the `config` folder (`config/application.yml`) and can be used to adjust Operate to your needs.

```
# Operate configuration file

camunda.operate:
  # Set operate userId, displayName and password.
  # If user with <userId> does not exists it will be created.
  # Default: demo/demo/demo
  userId: anUserId
  displayName: nameShownInWebpage
  password: aPassword
  roles:
    - OWNER
    - USER
  # ELS instance to store Operate data
  elasticsearch:
    # Cluster name
    clusterName: elasticsearch
    # Url
    url: http://localhost:9200
  # Zeebe instance
  zeebe:
    # Gateway address to zeebe
    gatewayAddress: localhost:26500
  # ELS instance to export Zeebe data to
  zeebeElasticsearch:
    # Cluster name
    clusterName: elasticsearch
    # url
    url: http://localhost:9200
    # Index prefix, configured in Zeebe Elasticsearch exporter
    prefix: zeebe-record
```

Logging

Tasklist uses Log4j2 framework for logging. In the distribution archive and inside a Docker image `/usr/local/tasklist/config/log4j2.xml`, logging configuration files are included and can be further adjusted to your needs:

```
<?xml version="1.0" encoding="UTF-8"?>
<Configuration status="WARN" monitorInterval="30">
  <Properties>
    <Property name="LOG_PATTERN">%clr{%d{yyyy-MM-dd HH:mm:ss.SSS}}{faint}
%clr{%5p} %clr{${sys:PID}}{magenta} %clr{---}{faint} %clr{[%15.15t]}{faint}
%clr{%-40.40c{1.}}{cyan} %clr{:}{faint} %m%n%xwEx</Property>
    <Property
name="log.stackdriver.serviceName">${env:TASKLIST_LOG_STACKDRIVER_SERVICENA
ME:-tasklist}</Property>
    <Property
name="log.stackdriver.serviceVersion">${env:TASKLIST_LOG_STACKDRIVER_SERVIC
EVERSION:-}</Property>
  </Properties>
  <Appenders>
    <Console name="Console" target="SYSTEM_OUT" follow="true">
      <PatternLayout pattern="${LOG_PATTERN}"/>
    </Console>
    <Console name="Stackdriver" target="SYSTEM_OUT" follow="true">
      <StackdriverLayout serviceName="${log.stackdriver.serviceName}"
        serviceVersion="${log.stackdriver.serviceVersion}" />
    </Console>
  </Appenders>
  <Loggers>
    <Logger name="io.camunda.tasklist" level="info" />
    <Root level="info">
      <AppenderRef ref="${env:TASKLIST_LOG_APPENDER:-Console}"/>
    </Root>
  </Loggers>
</Configuration>
```

By default, Console Appender is used.

JSON logging configuration

You can choose to output logs in JSON format (Stackdriver compatible). To enable it, define the environment variable `TASKLIST_LOG_APPENDER` like the following:

```
TASKLIST_LOG_APPENDER=Stackdriver
```

Change logging level at runtime

Tasklist supports the default scheme for changing logging levels as provided by Spring Boot.

The log level for Tasklist can be changed by following the Setting a Log Level section.

Set all Tasklist loggers to DEBUG

```
curl 'http://localhost:8080/actuator/loggers/io.camunda.tasklist' -i -X
POST \
-H 'Content-Type: application/json' \
-d '{"configuredLevel":"debug"}'
```

Web Modeler

Logging

Logging configuration for the `restapi` component

Web Modeler's `restapi` component uses the logback framework for logging. By default, the `restapi` component logs to the Docker container's standard output. To change the default logging behavior, create a custom configuration file and let the `restapi` know of it by specifying the following environment variable:

```
LOGGING_CONFIG=file:/full/path/to/custom-logback-config.xml
```

Refer to Spring Boot's logging documentation for more information on how to customize the logback configuration for specific use cases like logging to a file.

Enabling `DEBUG` logging for the `restapi` component can be useful for troubleshooting purposes, e.g. for debugging Zeebe connection issues.

By default, Web Modeler's `restapi` component logs in JSON. For a more readable logging format, activate the Spring profile using the following:

```
SPRING_PROFILES_INCLUDE=default-logging
```

Logging configuration for the `webapp` component

By default, the `webapp` component logs to the Docker container's standard output.

Logging to a file

To enable additional log output to a file, adjust the following environment variable:

```
LOG_FILE_PATH=/full/path/to/log/file.log
```

Configuring log levels

To control the verbosity of the logs, adjust the environment variables `LOG_LEVEL_CLIENT` (browser client) and `LOG_LEVEL_WEBAPP` (Node.js server).

```
LOG_LEVEL_CLIENT=DEBUG
```

The `LOG_LEVEL_*` options can be found here.

Logging configuration for the `websocket` component

By default, the `websocket` component logs to the Docker container's standard output.

Logging to a file

To enable additional log output to a file, follow these steps:

1. Mount a volume to the directory `/var/www/html/storage/logs`. The logs will be written to a file named `laravel.log` located inside this directory.
2. Adjust the following environment variable:

   ```
   LOG_CHANNEL=single
   ```

Zeebe Debug and Inspection tool

In the past, if Zeebe ran into some processing problems, there was no possibility to find out the last processing state. If there was no exporter configured or they haven't exported for a while, it was even worse since it was not clear what the last internal engine state was.

In order to shed some more light in the dark camunda build a tool called zdb — Zeebe Debugger. Zdb is a Java (17) CLI tool to inspect the internal state and log of a Zeebe partition. Now reaching version 1.8.0, with new features (printing and filtering the log in a nicer way).

`zdb` allows us to find the root cause, create fixes, and be prepared for the next upcoming (since failures always happen eventually). But also when investing in certain bugs. With `zdb`, we finally know what Zeebe was doing and how it came into that state.

Note: The output of `zdb` will always be JSON, which allows us to pipe it into `jq`, such that we can have nicer and filterable output. This is also used in our examples below.

General statistics

Often when you start working on an incident you need to get a first overview or understanding of what the state generally contains (depending on the problems of course). Here `zdb` can show you statistics of how many key-value pairs are stored in the internal state (in different column families).

```
$ zdb state --path <path-to-runtime-or-snapshot> | jq
{
  "DEFAULT": 1,
  "KEY": 1,
  "PROCESS_VERSION": 3,
  "PROCESS_CACHE": 3,
  "PROCESS_CACHE_BY_ID_AND_VERSION": 3,
  "PROCESS_CACHE_DIGEST_BY_ID": 3,
  "ELEMENT_INSTANCE_PARENT_CHILD": 6,
  "ELEMENT_INSTANCE_KEY": 6,
  "ELEMENT_INSTANCE_CHILD_PARENT": 6,
  "VARIABLES": 12,
  "TIMERS": 2,
```

```
  "TIMER_DUE_DATES": 2,
  "JOBS": 1,
  "JOB_STATES": 1,
  "JOB_DEADLINES": 1,
  "MESSAGE_START_EVENT_SUBSCRIPTION_BY_NAME_AND_KEY": 1,
  "MESSAGE_START_EVENT_SUBSCRIPTION_BY_KEY_AND_NAME": 1,
  "EVENT_SCOPE": 3,
  "EXPORTER": 2
}
```

An experienced Zeebe engineer or power user can see here already how many processes have been deployed, how many instances, jobs, timers, messages, etc. have been created and are currently in the state. This often helps to determine where to look next.

For example, if we see there are incidents in process instances in the state and the reported failure (ongoing incident) is about not progressing process instances we would check next the [open incidents]() in the state.

Restoring BPMN models

There are cases where you might lose your models, or you just want to find out which model has been currently deployed or is actually executed. Here `zdb` can help.

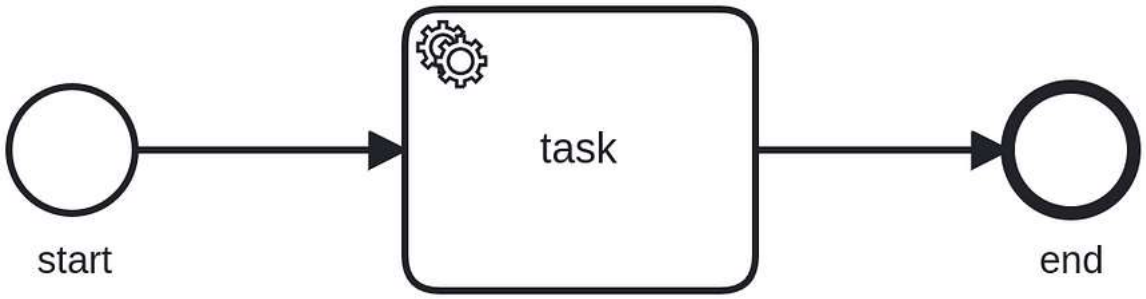
First, you can print all deployed process model metadata (it will show information like process definition key, version, and name).

```
zdb process list --path <path-to-runtime-or-snapshot> | jq
[
  {
    "bpmnProcessId": "benchmark",
    "resourceName": "bpmn/one_task.bpmn",
    "processDefinitionKey": 2251799813685363,
    "version": 1
```

```
  },
  {
    "bpmnProcessId": "timerProcess",
    "resourceName": "bpmn/timerProcess.bpmn",
    "processDefinitionKey": 2251799813685249,
    "version": 1
  },
  {
    "bpmnProcessId": "msg_one_task",
    "resourceName": "bpmn/msg_one_task.bpmn",
    "processDefinitionKey": 2251799813685581,
    "version": 1
  }
]
```

With a specific process definition key, we can print the complete process entity. Piping it here to `jq` allows us to filter for the resource, and the `--raw-output` option returns us the resource string without quotes. We can then direct the output to a file and have the model restored (you can open it with for example the [Camunda Modeler](#)).

```
zdb process entity 2251799813686656 --path <path-to-runtime-or-snapshot> \
|  jq --raw-output '.resource' > model.bpmn
```



Restored Model

Instances for a specific model

Sometimes you're interested in process instances of a specific process model.

You might have deployed a broken model and want to cancel all of the existing instances (that happened to us), but first, you need to find out all the keys of such instances.

You can use the following to print all instances for a certain process definition.

```
zdb process instances 2251799813685363 --path <path-to-runtime-or-snapshot> | jq
```

Printing the log

One of our most used `zdb` features is printing the entire log (default: as JSON).

```
zdb log print --path <path-to-log>
```

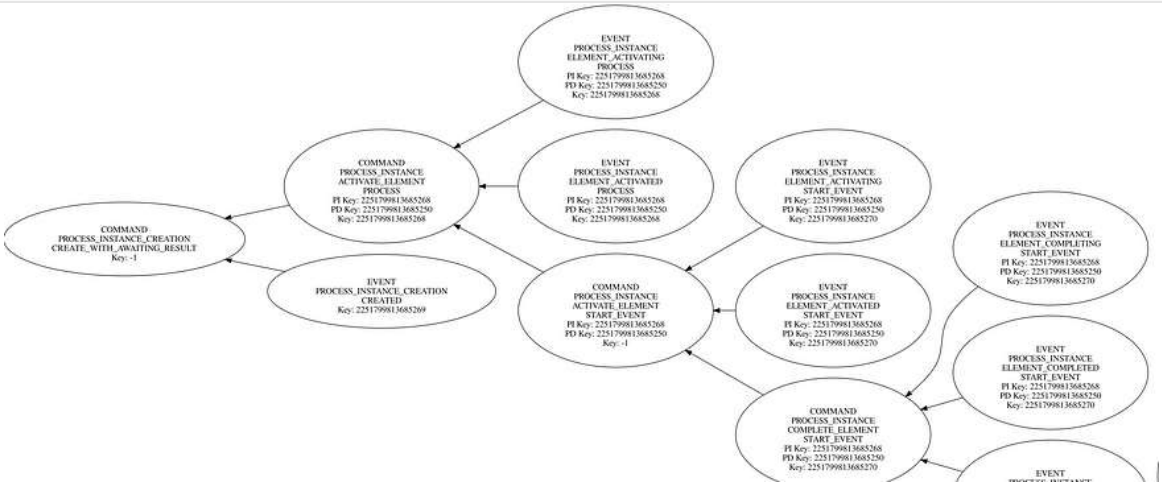
With the newest version (v1.8.0), `zdb` supports some built-in filters, like filtering for the process instance key. This means only records that correspond to a certain process instance are printed. Furthermore, we can limit the output now, with `--fromPosition` and `--toPosition` . You can read more about it here.

Not only JSON is the supported output format. `zdb` can print the log in dot format as well, which allows tracing commands.

```
zdb log print --format dot --path <path-to-log> > output.dot
```

With [Graphviz](#) you can visualize such dot files easily

```
dot -Tsvg -o output.svg output.dot
```



Trace of log

Printing and investigating the log is interesting since not all commands can be applied and are then not reflected in the state. The reasons can be many. Some might be rejected due to a wrong user input or wrong process instance state, etc. These commands and their rejections are still part of the log (if compaction hasn't happened yet) and can give you some interesting insights.

If you want to know more, check out the [GitHub repository.](#)

Troubleshooting

Keycloak requires SSL for requests from external sources

When deploying Camunda to a provider, it is important to confirm the IP ranges used for container to container communication align with the IP ranges Keycloak considers "local". By default, Keycloak considers all IPs outside those listed in their external requests documentation to be external and therefore require SSL.

As the Camunda Helm Charts currently do not provide support for the distribution of the Keycloak TLS key to the other containers, we recommend viewing the solution available in the Identity documentation.

Identity redirect URL

If HTTP to HTTPS redirection is enabled in the load-balancer or ingress, make sure to use the HTTPS protocol in the values file under `global.identity.auth.[COMPONENT].redirectUrl`. Otherwise, you will get a redirection error in Keycloak.

For example:

```
global:
  identity:
    auth:
    operate
      redirectUrl: https://operate.example.com
```

Zeebe Ingress (gRPC)

Zeebe requires an Ingress controller that supports `gRPC` which is built on top of `HTTP/2` transport layer. Therefore, to expose the Zeebe Gateway externally, you need the following:

1. An Ingress controller that supports `gRPC` (ingress-nginx controller supports it out of the box).
2. TLS (HTTPS) via Application-Layer Protocol Negotiation (ALPN) enabled in the Zeebe Gateway Ingress object.

However, according to the official Kubernetes documentation about Ingress TLS:

There is a gap between TLS features supported by various Ingress controllers. Please refer to documentation on nginx, GCE, or any other platform specific Ingress controller to understand how TLS works in your environment.

Therefore, if you are not using the ingress-nginx controller, ensure you pay attention to TLS configuration of the Ingress controller of your choice. Find more details about the Zeebe Ingress setup in the Kubernetes platforms supported by Camunda.

Identity `contextPath`

Camunda 8 Self-Managed can be accessed externally via different methods. One such method is the combined Ingress setup. In that configuration, Camunda Identity is accessed using a specific path, configured by setting the `contextPath` variable, for example `https://camunda.example.com/identity`.

For security reasons, Camunda Identity requires secure access (HTTPS) when a `contextPath` is configured. If you want to use Camunda Identity with HTTP, use a separate Ingress setup (applications such as Operate, Optimize, etc, can still be accessed in a combined setup).

NOTE

Due to limitations, the Identity `contextPath` approach is unavailable when using a browser in Incognito mode.

Web Modeler database schema

The Web Modeler `restapi` component requires a database connection. This connection should not point to the same database as Keycloak does.

Gateway timeout on redirect

A gateway timeout can occur if the headers of a response are too big (for example, if a JWT is returned as `Set-Cookie` header). To avoid this, you can increase the `proxy-buffer-size` of your Ingress controller or Ingress. The setting for **ingress-nginx** can be found here.

Helm CLI version and installation failures

If you encounter errors during Helm chart installation, such as type mismatches or other template rendering issues, you may be using an outdated version of the Helm CLI. Helm's handling of data types and template syntax can vary significantly between versions. Ensure you use the Helm CLI version `3.13` or higher.

DNS disruption issue for Zeebe in Kubernetes clusters (1.29-1.31)

Kubernetes clusters running versions 1.29 to 1.31 may experience DNS disruptions during complete node restarts, such as during upgrades or evictions, particularly if the cluster's DNS resolver pods are affected.

This issue is specifically noticeable for Zeebe (Netty), as it will no longer be able to form a cluster because of improper DNS responses. This occurs because Zeebe continues to communicate with a non-existent DNS resolver, caused by improper cleanup of conntrack entries for UDP connections.

Details on this issue can be found in this Kubernetes issue and has been resolved in the following patch releases:

- Kubernetes 1.29.10
- Kubernetes 1.30.6
- Kubernetes 1.31.2

Kubernetes versions 1.32 and versions before 1.29 are not affected.

If an immediate cluster upgrade to a fixed version is not possible, the following temporary workarounds can be applied if you encounter DNS issues:

- Restart the `kube-proxy` pod(s)
- Delete the affected Zeebe pod

Anomaly detection scripts

The c8-sm-checks project introduces a set of scripts to aid detection of Camunda deployment anomalies.

These scripts perform health checks on various aspects of the Kubernetes installation and Zeebe components, providing insights into potential issues that may affect the performance or stability.

Usage

Each script in the `c8-sm-checks` project can be executed independently, allowing you to target specific areas for troubleshooting and verification.

To utilize these scripts effectively, ensure you have the necessary permissions and access to your Kubernetes cluster. Additionally, make sure you have the required dependencies installed on your system, such as `kubectl`, `helm`, `curl`, and `grpcurl`.

For detailed documentation and usage instructions for each script, refer to the [c8-sm-checks GitHub repository]. Additionally, you can use the `-h` option with each script to display help information directly from the command line.

Before using it, clone the `c8-sm-checks` repository to your local environment by running the following command:

```
git clone https://github.com/camunda/c8-sm-checks.git
cd c8-sm-checks
```

Kubernetes connectivity scripts

These scripts enable you to verify the connectivity and configuration of your Kubernetes cluster, including checks for deployment status, service availability, and Ingress configuration.

Kubernetes permissions

When utilizing the anomaly detection scripts within a Kubernetes environment, ensure the user has specific permissions:

- **List pods**: Required for `kubectl get pods` to fetch pod details in the namespace.
- **Execute commands in pods**: Necessary for running commands inside pods via `kubectl exec`.
- **List services**: Needed for `kubectl get services` to retrieve service information.
- **List ingresses**: Required by `kubectl get ingress` to obtain Ingress objects.
- **Get Ingress details**: Necessary for `kubectl get ingress` to fetch Ingress configurations.

Deployment check (`./checks/kube/deployment.sh`)

This script checks the status of a Helm deployment in the specified namespace, ensuring that all required containers are present and ready. You can customize the list of containers to check based on your deployment topology.

```
./checks/kube/deployment.sh -n camunda-primary -d camunda -c "zeebe,zeebe-gateway,web-modeler"
```

Connectivity check (`./checks/kube/connectivity.sh`)

This script verifies Kubernetes connectivity and associated configuration, checking for the presence of services and ingresses that conform to the required specifications.

```
./checks/kube/connectivity.sh -n camunda-primary
```

Zeebe connectivity scripts

These scripts focus on verifying the connectivity and health of Zeebe components within your deployment. You can check token generation, gRPC connectivity, and other essential aspects of your Zeebe setup.

gRPC Zeebe check (`./checks/zeebe/connectivity.sh`)

This script verifies connectivity to a Zeebe instance using HTTP/2 and gRPC protocols, providing insights into the health and status of your Zeebe deployment.

```
./checks/zeebe/connectivity.sh -a https://local.distro.example.com/auth/realms/camunda-platform/protocol/openid-connect/token -i myclientid -s 0Rn28VrQxGNxowrCWe6wbujwFghO4990 -u zeebe.distro.example.com -H zeebe.local.distro.example.com:443
```

Find more information on [how to register your application on Identity].

IRSA configuration check

The AWS EKS IRSA configuration scripts are focused on verifying the correct setup of IAM Roles for Service Accounts (IRSA) within your Kubernetes deployment on AWS. These scripts ensure that your Kubernetes service accounts are correctly associated with IAM roles, allowing components like PostgreSQL, OpenSearch, and others in your deployment to securely interact with AWS resources.

For detailed usage instructions and setup information, please refer to the [IRSA guide].

Interpretation of the results

Each script produces an output indicating the status of individual checks, which can be either `[OK]`, which signals a healthy status, or `[FAIL]`, which signals an unhealthy status.

While the scripts continue execution even if a check fails, it may be necessary to review the logs to identify the failed element.

At the end of each script, a global check status is provided, indicating whether any tests failed and the corresponding error code. For example:

```
[FAIL] ./checks/zeebe/connectivity.sh: At least one of the tests failed
(error code: 5).
```

Handling errors

If a check fails, it indicates a deviation from the expected configuration on a normal setup. Resolving the error involves studying the failed check and applying the best practices outlined in the documentation (use the search feature to find the associated recommendation for a failed check).

For example:

```
[FAIL] None of the ingresses contain the annotation
nginx.ingress.kubernetes.io/backend-protocol: GRPC, which is required for
Zeebe ingress.
```

The error message suggests adjusting the Ingress configuration to include the required annotation. One can also explore the source of the script to have a better understanding of the reason for the failure.

NOTE

Sometimes, some checks may not be applicable to your setup if it's custom (for example, with the previous example the Ingress you use may not be ingress-nginx).